Generic Trees

October 23, 2026

0.1 Initializing the Tree

[1]:
```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []

root = TreeNode(1)

child1 = TreeNode(2)
child2 = TreeNode(3)
child3 = TreeNode(4)

root.children.append(child1)
root.children.append(child2)
root.children.append(child3)

print(root.children[0].data)
```

```
2
```

0.2 Printing the Tree Nodes

[ ]:
```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []

root = TreeNode(1)

child1 = TreeNode(2)
child2 = TreeNode(3)
child3 = TreeNode(4)
child4 = TreeNode(5)

root.children.append(child1)
root.children.append(child2)
root.children.append(child3)
```

```
root.children.append(child4)

def print_tree(root):
    print(root.data)
    for childNode in root.children:
        print_tree(childNode)

print_tree(root)
```

```
1
2
3
4
5
```

0.3 Printing the tree in the better way

[8]:
```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []

root = TreeNode(1)

child1 = TreeNode(2)
child2 = TreeNode(3)
child3 = TreeNode(4)
child4 = TreeNode(5)
child5 = TreeNode(6)


root.children.append(child1)
root.children.append(child2)
root.children.append(child3)
root.children.append(child4)
root.children.append(child5)

def print_tree_better(root):
    if root is None:
        return

    print(root.data, end=":")

    for eachChild in root.children:
        print(eachChild.data, end=",")

    print()
```

```
    for eachChild in root.children:
        print_tree_better(eachChild)


print_tree_better(root)

child2.children.append(child3)
child2.children.append(child5)

child4.children.append(child3)
child4.children.append(child2)

print_tree_better(root)
```

```
1:2,3,4,5,6,
2:
3:
4:
5:
6:
1:2,3,4,5,6,
2:
3:4,6,
4:
6:
4:
5:4,3,
4:
3:4,6,
4:
6:
6:
```

0.4 Take the Input recursivilly

[11]:
```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []

def print_tree_better(root):
    if root is None:
        return

    print(root.data, end=":")
```

```
    for eachChild in root.children:
        print(eachChild.data, end=",")

    print()

    for eachChild in root.children:
        print_tree_better(eachChild)


def take_input():
    data  = int(input("Enter the data for the node: "))

    Node = TreeNode(data)

    num_child = int(input(f"Enter the no of child for the {data}: "))

    for _ in range(num_child):
        child = take_input()
        Node.children.append(child)

    return Node

root = take_input()

print_tree_better(root)
```

```
1:2,6,7,
2:3,
3:4,5,
4:
5:
6:8,9,
8:
9:
7:10,
10:
```

0.5 Take input Level Wise

[ ]:
```
from collections import deque

class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = []

def print_tree_better(root):
```

```
    if root is None:
        return
    print(root.data, end=":")
    for eachChild in root.children:
        print(eachChild.data, end=",")
    print()
    for eachChild in root.children:
        print_tree_better(eachChild)

def take_input_level_wise():
    data = int(input("Enter the root Node data: "))
    root = TreeNode(data)

    queue = deque([root])

    while queue:
        current_node = queue.popleft()

        num_children = int(input(f"Enter the number of Children for␣
 ↪{current_node.data}: "))

        for i in range(num_children):
            child_data = int(input(f"Enter the data for child {i+1} of Node␣
 ↪{current_node.data}: "))
            child_node = TreeNode(child_data)
            current_node.children.append(child_node)
            queue.append(child_node)

    return root


root = take_input_level_wise()
print("\n--- Final Tree Structure ---")
print_tree_better(root)
```

```
--- Final Tree Structure ---
1:2,6,7,
2:2,
2:6,
6:
6:1,
1:4,3,2,3,4,
4:
3:
2:
3:
```

```
4:
7:10,
10:6,7,8,0,0,
6:
7:
8:
0:
0:
```

0.6 Count Nodes in Tree

[2]:
```python
class GenericTree:
    def __init__(self, data):
        self.data = data
        self.children = []


def predifined_generic_tree_input():
    root1 = GenericTree(1)
    child1 = GenericTree(2)
    child2 = GenericTree(3)

    root1.children.append(child1)
    root1.children.append(child2)

    child1.children.append(GenericTree(4))
    child1.children.append(GenericTree(5))

    return root1

def print_tree_better(root):
    if root is None:
        return
    print(root.data, end=":")
    for eachChild in root.children:
        print(eachChild.data, end=",")
    print()
    for eachChild in root.children:
        print_tree_better(eachChild)


def count_tree_nodes(root):
    if root==None:
        return 0

    noofnodes = 1
```

```
    for eachchild in root.children:
        noofnodes+=count_tree_nodes(eachchild)

    return noofnodes

root1 = predifined_generic_tree_input()

print_tree_better(root1)

print("printing the nodes of the tree: ")
print(count_tree_nodes(root1))
```

```
1:2,3,
2:4,5,
4:
5:
3:
printing the nodes of the tree:
5
```

0.7 Hight of a Tree

[8]:
```
class Tree:
    def __init__(self, data):
        self.data = data
        self.children = []

def myTree():
    root = Tree(1)
    ch1 = Tree(2)
    ch2 = Tree(3)
    ch3 = Tree(4)
    ch4 = Tree(5)

    root.children.append(ch1)
    root.children.append(ch2)

    ch2.children.append(ch3)
    ch2.children.append(ch4)

    ch3.children.append(ch1)

    return root


def print_tree(root):
    if root== None:
```

```
        return
    print(root.data, end=":")
    for child in root.children:
        print(child.data, end=",")

    print()
    for child in root.children:
        print_tree(child)


def height_of_tree(root):
    if root==None:
        return 0

    hight = 1
    max_child_hight = 0

    for child in root.children:
        max_child_hight = max(max_child_hight, height_of_tree(child))

    hight = hight+max_child_hight

    return hight

root1 = myTree()

print_tree(root1)

print("height of the tree is: ", height_of_tree(root1))
```

```
1:2,3,
2:
3:4,5,
4:2,
2:
5:
height of the tree is:  4
```

0.8 Traversal in Tree

[17]:
```
class Tree:
    def __init__(self, data):
        self.data = data
        self.children = []

def myTree():
    root = Tree(1)
    ch1 = Tree(2)
```

```
    ch2 = Tree(3)
    ch3 = Tree(4)
    ch4 = Tree(5)
    ch5 = Tree(6)
    root.children.append(ch1)
    root.children.append(ch2)

    ch2.children.append(ch3)
    ch2.children.append(ch4)

    ch3.children.append(ch5)

    return root


def print_tree(root):
    if root== None:
        return
    print(root.data, end=":")
    for child in root.children:
        print(child.data, end=",")

    print()
    for child in root.children:
        print_tree(child)


def preorder_traversal(root):
    if root is None:
        return
    print(root.data, end=" ")
    for child in root.children:
        preorder_traversal(child)

def post_order_traversal(root):
    if root is None:
        return

    for each in root.children:
        post_order_traversal(each)

    print(root.data, end=" ")



root1 = myTree()
print("Your Tree is: ")
```

```
print_tree(root1)

print("pre oredre of the tree is: ")
preorder_traversal(root1)
print()
print("Post order of the tree is: ")
post_order_traversal(root1)
```

```
Your Tree is:
1:2,3,
2:
3:4,5,
4:6,
6:
5:
pre oredre of the tree is:
1 2 3 4 6 5
Post order of the tree is:
2 6 4 5 3 1
```

0.9 Sum of Nodes

[ ]:
```
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []

def sum_of_nodes(root):
    """
    Function to find the sum of all nodes in an N-ary tree.
    :param root: Node -> The root of the N-ary tree
    :return: int -> The sum of all node values in the tree
    """
    if root is None:
        return 0
    ans = root.val

    for each in root.children:
        ans+= sum_of_nodes(each)

    return ans
```